

DigitalOcean Reference Build

What We Built, What It Costs, and How to Present It to Customers

Prepared 2026-09-30 | Account: sprabhakar@digitalocean.com

1. What We Built

A complete, working reference implementation on DigitalOcean: a web application connected to two managed databases.

Component	Resource / Tier	Where
Web app	my-app (Node.js/Express)	nyc
Relational DB	my-app-pg (PostgreSQL 18)	nyc1
Cache / store	my-app-valkey (Valkey 8)	nyc1

Live URL: <https://my-app-2x7kf.ondigitalocean.app>

Source code: <https://github.com/srihariprabhakar/my-app>

The app exposes three endpoints:

- GET / -> liveness check (status: ok)
- GET /health -> reports both databases as "ok"
- POST /visit -> writes a note to PostgreSQL and increments a Valkey counter

Verified working: the health check returns postgres: ok and valkey: ok, and each /visit call persists data and returns a running count.

2. Purpose

This build is a reference architecture, not a finished product. It exists to prove three things for a customer conversation:

- That a stateless API can be stood up safely on App Platform in minutes.
- That a relational database (PostgreSQL) and a cache (Valkey) connect cleanly and are reachable from the app.
- That the data path (write + read) is durable and verifiable end-to-end.

It is the starting skeleton a customer would build a real product on, by swapping in their own routes, schema, and domain.

3. What It Costs

Approximate monthly list prices for this exact configuration:

Resource	Tier	USD / mo
App Platform service	basic-xxs (1 instance)	~\$5
Managed PostgreSQL	db-s-1vcpu-1gb	~\$15
Managed Valkey	db-s-1vcpu-1gb	~\$15
Total		~\$35

Build/deploy costs are negligible - roughly 11-13 billable seconds per deploy, a fraction of a cent. Only the running resources generate steady monthly billing.

Note: figures are approximate U.S. list prices; confirm engine/version pricing in the billing dashboard.

4. How to Present This to a Customer

Use this narrative to validate a customer build. Lead with the outcome, then the "why" behind each choice.

Opening line (the value):

- "Your app is live, your data is saved, and everything is connected and verified - so you can trust this foundation before writing business logic."

Three proof points to walk through:

- Working, not promised - show the live URL and the /health response with "ok" for both databases.
- Durable data - run /visit and show a row actually lands in PostgreSQL and the counter increments, so nothing is vaporware.
- Ready to scale - explain that the same setup can grow from a ~\$35/mo dev tier to production sizes without re-architecting.

Address reliability honestly:

- Managed databases mean automated backups, patching, and snapshots are built in - no admin for them to run.
- App Platform gives zero-downtime deploys, automatic HTTPS, health checks, and crash restarts.
- Be transparent that the databases are single-node today; recommend a standby/HA tier for mission-critical use, and right-sizing the dev tiers before launch.

Close with the next step:

- "We have the skeleton; next we add your schema, your endpoints, and your domain - and scale it as traffic grows."

5. Reliability & Scaling (Quick Reference)

- Scale vertically first (bigger instance_size_slug), then horizontally (add instances - App Platform auto-balances).
- Add connection pooling (PgBouncer / pg pool) as concurrency grows.
- Enable a standby node or HA tier on databases for mission-critical loads.
- Add alerting (CPU/memory thresholds) - currently only DEPLOYMENT_FAILED is set.
- All resources share one region (nyc / nyc1), keeping app-to-DB latency low.